SkillSphere — Deep Theoretical Explanation

1. What Is SkillSphere and Why Does It Exist?

SkillSphere is a **career intelligence platform** — a web application that sits at the intersection of three real-world problems:

1. **Students and job-seekers don't know what to learn** to get a particular job.
2. **Companies waste time** manually screening resumes and managing hiring pipelines.
3. **Profiles scattered across platforms** (LinkedIn, GitHub, resume PDFs) are hard to consolidate into one professional identity.

SkillSphere tries to solve all three in one place:

- **For candidates**: build a structured profile, get an AI-generated learning roadmap, browse and apply to jobs.
- **For companies**: post jobs, receive applications, and move candidates through a hiring pipeline.

---

2. The Big-Picture Architecture

Before going deep, understand the high-level split:

```
[ Browser (React App) ]  ↔  [ Node.js / Express Server ]  ↔  [ MongoDB Database ]
                                          ↑
                                [ External Services ]
                                - Firebase Auth
                                - Groq AI (Llama 3.3 70B)
                                - Gemini AI (2.0 Flash)
                                - Zoho SMTP (Email)
                                - GitHub API
```

The app is a **classic client-server architecture** with a twist — it uses **two authentication systems working together** (Firebase + JWT), and **two AI models** for different tasks. Every decision here was made deliberately, and we'll explain the "why" throughout.

---

3. The Frontend — Theory of How the React App Works

3.1 React and the "Single Page Application" Concept

The entire frontend is built with **React**, which means the browser loads exactly **one HTML file** (`index.html`) when the user visits the site. After that, JavaScript takes over and renders every page entirely in the browser — no server sends new HTML pages when you navigate. This is called a **Single Page Application (SPA)**.

The benefit: navigation is instant (no page reloads). The challenge: you have to manage routing, state, and loading entirely yourself in JavaScript.

3.2 Vite — Why It's the Build Tool

Vite is the tool that:

- Runs the development server (hot-reloads changes instantly)
- Bundles all React/JSX/CSS files into optimized JavaScript for production

Think of Vite as the engine room of the frontend — you never interact with it directly, but everything depends on it.

3.3 React Router — How Navigation Works

When you click a link inside the app (like going from Dashboard to Jobs), React Router **intercepts** that click. Instead of asking the server for a new page, it simply **swaps out which React component is rendered on screen** — all without a full reload.

The app defines its routing tree in `App.jsx`. There are three key concepts here:

Routes — map a URL path to a React component:

- `/` → `HomePage`
- `/signin` → `SignInPage`
- `/dashboard/candidate` → `StudentDashboardPage`
- etc.

Nested Routes — the sidebar and topbar are rendered once as a "layout" (`CandidateLayout` or `CompanyLayout`), and the actual page content swaps inside it via `<Outlet/>`. This is why navigating between candidate pages (Dashboard → Jobs → Roadmap) does not cause the sidebar to flicker or re-render — it was already there.

Route Guards — before rendering a page, the app checks conditions:

- If the user is not logged in → send them to `/signin`
- If the user is a candidate but hasn't completed their profile → send them to `/profile-builder`
- If a company tries to visit a candidate-only route → send them to home

This is the `ProtectedRoute` component in `App.jsx`. It reads from the global auth state and redirects accordingly.

3.4 React Context — Global State Management

React passes data between components using "props" (parent → child), but this breaks down when many unrelated components need the same data (e.g., the user object is needed in the Sidebar, the Topbar, every page, every route guard). Passing it as props through every level would be messy — this is called "prop drilling."

React Context solves this by creating a "global store" that any component can read from directly.

SkillSphere has three contexts:

`AuthContext` — the most critical one. It stores:

- `user` — the signed-in user object (or null if not signed in)
- `loading` — whether the auth check is still running
- `refreshUser()` — a function to re-fetch the user from the server

When the app first loads, `AuthContext` immediately checks if there's a stored access token (in `localStorage`). If yes, it calls the backend's `/api/user/me` endpoint to get the full user object. Until that check completes, `loading` is `true` and the splash screen is shown. This is how the app knows whether you're logged in without making you log in every time.

`JobsContext` — manages the jobs list for candidates (fetching, filtering, pagination).

`RoadmapContext` — manages the AI-generated career roadmap (current roadmap, previous roadmap, generation state).

3.5 The API Layer (`services/api.js`) — How the Frontend Talks to the Backend

Every HTTP request the frontend makes goes through one single file: `api.js`. This is a deliberate architectural choice — instead of scattering `fetch()` calls across components, one central place handles everything.

This file uses **Axios**, a library that simplifies HTTP requests.

Three critical things this file does:

① **Automatic Token Attachment** Every request that goes out automatically has `Authorization: Bearer <accessToken>` added to its headers. The component making the request doesn't need to know about tokens at all. This is done using Axios's "request interceptor" — it runs a function before every single request.

② **Automatic Token Refresh (Silent Re-authentication)** Access tokens expire after 7 days. When a request fails with a `401 Unauthorized` error, instead of logging the user out immediately, the interceptor:

1. Takes the stored refresh token
2. Silently calls `/api/auth/refresh` to get a new access token
3. **Retries the original failed request** with the new token
4. The user never sees anything — it just works

If multiple requests fail simultaneously while a refresh is already in progress, they're queued up (`failedQueue`) and all retried once the new token arrives — this prevents multiple simultaneous refresh calls.

If the refresh itself fails (token expired), only then does it clear everything and redirect to `/signin`.

③ **A Single Source of Truth for All API Calls** Every backend operation (`login`, `getJobs`, `saveProfile`, `generateRoadmap`, etc.) is a named exported function in this file. Components import and call these functions — they never write raw Axios calls. This makes the entire data layer easy to change in one place.

---

4. The Pages — What Each One Does and Why

4.1 The Landing Page (`HomePage.jsx`)

This is the **public-facing marketing page**. Its job is to communicate what SkillSphere is, convince visitors to sign up, and look visually impressive. It uses **Framer Motion** for scroll animations — elements fade and slide in as you scroll down.

Theoretically, a landing page is a **conversion funnel** — it moves the visitor from "curious" to "clicking Sign Up."

4.2 Authentication Pages

There are three auth pages, each solving a specific problem:

`GetStartedPage` **(Sign Up) — 3-Step Flow** This is not a simple form. Sign-up is a 3-step wizard:

1. Enter email → backend sends an OTP to that email
2. Enter OTP → backend verifies it (but still hasn't created the account)
3. Enter name, password, choose role → backend creates the account and issues tokens

Why 3 steps instead of just a sign-up form? **Email verification without a separate "click the link" step**. The OTP approach verifies the email during registration itself, blocking fake/throwaway emails. The account is only created after a verified email — this prevents spam accounts.

`SignInPage` Supports three ways to sign in:

- Email/password
- Google OAuth
- GitHub OAuth

Also has a **role-aware toggle** (Candidate vs Company). Why? The same email can theoretically have different roles, and the backend enforces that you sign in with the role your account was registered as.

`ForgotPasswordPage` **— Another 3-Step Flow**

1. Enter email → backend silently sends OTP (if the email exists; it doesn't reveal whether it does or not — this prevents "email enumeration attacks" where someone probes which emails are registered)
2. Enter OTP → backend returns a short-lived "reset token" (valid for 15 minutes)
3. Enter new password → backend updates Firebase + MongoDB using the reset token

4.3 Profile Builder (`ProfileBuilderPage`)

This is the most complex page in the candidate flow. It's a **multi-section wizard** that collects:

- Personal info (name, title, email, phone, location, portfolio, LinkedIn, GitHub, summary, photo)
- Education (multiple entries)
- Work Experience (multiple entries)
- Projects (multiple entries)
- Skills (4 categories: Languages, Frameworks, Tools, Libraries)
- Certifications (with optional PDF upload)
- Awards
- Leadership positions
- Volunteer work
- Publications

- Extra achievements and interests
- Consent flags (whether the profile can be stored and shared with recruiters)

Why is this a separate "builder" and not just an "edit profile" page?

Theoretically, new users need guidance. The profile builder is a **guided onboarding wizard** that walks them through every section step-by-step. It:

- Auto-saves every 30 seconds (prevents losing data)
- Shows a progress indicator (motivates completion)
- Only unlocks the dashboard when complete (the `profileCompleted` flag on the User model)

This design pattern is called a **progressive disclosure wizard** — you don't overwhelm the user with everything at once; you break it into manageable sections.

4.4 The Candidate Dashboard (`StudentDashboardPage`)

This is the candidate's "home base" after login. Theoretically, a dashboard is a **summary view** — it doesn't deep-dive into any one feature, but gives you a snapshot of everything:

- Profile completion percentage
- Your GitHub repositories (pulled from the GitHub cache in your profile)
- Your current career roadmap (brief summary)
- Recently posted jobs
- Your skills overview

The GitHub section is interesting: when you first visit, the backend checks if your profile has cached repos. If yes, it serves them instantly. If no, it fetches from the GitHub API live. This **cache-first strategy** ensures the dashboard always loads quickly.

4.5 Career Roadmap Page (`CareerRoadmapPage`)

This is the AI-powered centerpiece for candidates. The user types a target job role (e.g., "DevOps Engineer") and hits generate. The backend calls Groq's API with a highly engineered system prompt, which instructs the AI to return a **structured JSON roadmap** with exactly 4 phases.

☑ — where do I start?" problem. Instead of spending hours researching, a learner gets a structured, phase-by-phase curriculum in seconds.

The "load previous roadmap" feature acts as an **undo** — you can always go back to what you had before generating a new one.

4.6 Jobs Page (`JobsPage`)

The candidate's job board. It fetches all active job listings from the backend with support for filtering (employment type, workplace type, location, salary range). Clicking a job opens a detail view, and clicking "Apply" opens the Application Modal.

4.7 Profile Page (`ProfilePage`)

The candidate's full profile in read/edit mode — a slightly different version of the profile builder, designed for ongoing edits rather than first-time setup. The candidate can update any section at any time.

4.8 Company Pages

`CompanyDashboardPage` — stats overview (total postings, applicants, active jobs).

`CompanyProfilePage` — the company can set its name, description, logo, website URL, LinkedIn, and Twitter. This data appears on job postings so candidates can see who's hiring.

`PostJobPage` — a long, structured form to create or edit a job posting. Jobs can be saved as a **draft** (visible only to the company) and later **published** (visible to candidates). Supports optional PDF attachments (job description, company brochure).

`ApplicationsPage` — the company's view of all candidates who have applied to any of their jobs. Shows each candidate's info, their pitch, resume source, and allows the recruiter to move them through the pipeline stages.

`CandidatesPage` — a browsable list of candidates who have applied.

---

5. The Backend — Theory of How the Server Works

5.1 What a Backend Server Actually Does

The backend is a **REST API server** — it listens on a network port (5000), waits for HTTP requests, processes them, and sends back JSON responses. It never sends HTML pages; that's the React frontend's job.

The server is built with **Express**, which is a framework that makes it easy to:

- Define routes (`GET /api/jobs`, `POST /api/auth/signup`, etc.)
- Apply middleware (functions that run before your route handler)
- Send consistent responses

5.2 The Middleware Stack — The Request Pipeline

Every HTTP request that arrives at the backend passes through a **pipeline of middleware functions**, in order, before reaching the actual route handler.

In `app.js`, this pipeline is:

① **Helmet** — sets security-related HTTP headers automatically. For example, it tells browsers not to sniff the MIME type of responses and blocks clickjacking. Think of it as a security checklist automatically applied to every response.

② **CORS** — "Cross-Origin Resource Sharing." Browsers have a security rule: JavaScript from `http://localhost:5173` (Vite dev server) is not allowed to make requests to `http://localhost:5000` (Express server) unless the server explicitly says it's OK. The CORS middleware adds the `Access-Control-Allow-Origin` header to tell the browser: "Yes, requests from 5173 are allowed." Without this, the browser would block all API calls.

③ **Body Parsing** — when a request arrives with a JSON body, Express can't read it by default. `express.json()` parses the raw bytes into a JavaScript object so you can do `req.body.email` instead of manually parsing bytes.

④ **Morgan** — logs every incoming request to the console in development. Useful for debugging (you can see exactly what requests are hitting the server and in what order).

⑤ **Rate Limiters** — there are two:

- **Global limiter**: 600 requests per 15 minutes per IP. This is a backstop against DDoS (someone flooding the server with requests to overwhelm it).
- **Auth limiter**: 30 requests per 15 minutes, applied only to `/api/auth/*`. This is a tighter limit to prevent brute-force attacks (trying millions of passwords). Auth endpoints are the most sensitive so they get a dedicated, stricter limit.

⑥ **Routes** — after all middleware, the request reaches its specific route handler.

5.3 Module Structure — Why It's Organized This Way

The backend is organized into **feature modules**, each in its own folder:

```
auth/       — authentication
user/       — user account management
profile/    — candidate profile
github/     — GitHub repo fetching
roadmap/    — AI roadmap generation
job/        — job postings and applications
```

Each module follows the same **3-layer pattern**:

- **Routes** (`*.routes.js`) — defines what URL + method maps to what handler; applies middleware
- **Controller** (`*.controller.js`) — receives the request, calls the service, sends the response
- **Service** (`*.service.js`) — contains the actual business logic (database queries, external API calls, calculations)

Why this separation? **Single Responsibility Principle**. Each layer has one job. The route doesn't know how the business logic works. The controller doesn't know the database schema. The service doesn't know about HTTP. This makes the code easier to test, modify, and understand.

---

6. Authentication — The Dual-System Theory

This is the most complex architectural decision in the project. SkillSphere uses **two authentication systems simultaneously**: Firebase Auth and JWT. Understanding why requires understanding each one.

6.1 What Firebase Auth Does

Firebase Auth is Google's cloud authentication service. When a user signs in with email/password (or Google/GitHub OAuth), **Firebase handles all the credential verification**:

- It checks if the password is correct
- It manages Google/GitHub OAuth token exchange
- It manages password reset for OAuth flows
- It returns an **ID token** (a short-lived JWT signed by Google)

The key thing: SkillSphere's backend **never sees or stores the user's password** for email/password accounts. Firebase stores and verifies it. This is a major security advantage — even if SkillSphere's database were compromised, no passwords would be leaked.

6.2 What SkillSphere's Own JWT System Does

Firebase's ID tokens expire in 1 hour and require a call to Google to verify them. Making every API request verify a Firebase token would be:

- Slow (network call to Google for every request)
- A dependency on Google's uptime
- Inflexible (can't store custom claims like MongoDB user ID and role)

So instead, after Firebase verification (during sign-in), the backend **issues its own JWT tokens**:

- **Access Token** — valid for 7 days, contains `{ userId, email, role }`
- **Refresh Token** — valid for 30 days, used to get new access tokens

All subsequent API calls use SkillSphere's own access token, not Firebase's. The backend verifies these locally using the `JWT_SECRET` key — no network call needed.

6.3 How They Work Together

```
SIGN UP (Email):
User fills form → frontend sends email to backend
Backend: email not taken → send OTP via Zoho SMTP
User enters OTP → backend verifies OTP in DB
User submits name/password/role →
  backend calls Firebase Admin SDK: auth.createUser(email, password)
  Firebase creates the account, returns uid
  backend creates MongoDB User doc with that uid
  backend issues its own access + refresh tokens
  frontend stores tokens in localStorage

SIGN IN (Email):
User enters email/password →
  Firebase CLIENT SDK (in the browser) verifies the password directly with Google
  Firebase returns an ID token to the browser
  frontend sends ID token to backend's /api/auth/signin
  backend middleware calls firebase-admin: auth.verifyIdToken(idToken) — validates
the signature
  backend finds User in MongoDB
  backend issues its own access + refresh tokens
  frontend stores tokens

ALL SUBSEQUENT API CALLS:
```

```
Frontend attaches: Authorization: Bearer <accessToken>
Backend middleware: verifyAccessToken(token) — checks JWT signature locally (no
network call)
  → attaches user to req.user
  → route handler runs

TOKEN EXPIRY:
Access token expires → API returns 401
api.js interceptor catches 401 → calls /api/auth/refresh with refresh token
Backend: verifyRefreshToken → find user → bcrypt.compare(token, stored hash)
  → issue new access + refresh tokens
api.js: retries the original failed request
User never notices anything
```

6.4 Security Details

Refresh token hashing: The refresh token itself is stored in MongoDB **bcrypt-hashed** (not in plain text). So even if the database were leaked, the refresh tokens couldn't be used — an attacker would need to brute-force them.

OTP hashing: Similarly, OTP codes are **bcrypt-hashed** in the database. Only the real 6-digit code sent to the email can match.

Token rotation: Every time a refresh is used, **both** access and refresh tokens are re-issued. The old refresh token is replaced (previous hash overwritten). This means a stolen refresh token has a limited window to be used.

---

7. The Database — Theory of MongoDB and the Data Models

7.1 Why MongoDB (Not a Relational Database)?

MongoDB is a **document database** — instead of rows and columns (like MySQL), it stores **JSON-like documents** in collections. Each document can have a different shape.

Why choose MongoDB for SkillSphere?

- A candidate's profile has a variable number of education entries, experiences, projects, certifications, etc. In a relational database, each of these would be a separate table with foreign keys. In MongoDB, they're all **embedded inside one profile document** as arrays.
- Job postings embed their entire `applications[]` array inside the job document itself — no join needed to check if a candidate has applied.
- Document shape flexibility means the schema can evolve without database migrations.

7.2 The Data Models

User Model — the core identity:

```
email, role ('candidate' or 'company'), fullName/companyName,
photoURL, provider ('email'/'google'/'github'), firebaseUid,
```

```
isVerified, isActive, refreshToken (hashed), profileCompleted,
socialLinks (linkedin, twitter)
```

One User document per account. The `profileCompleted` flag is critical — it gates the entire candidate experience.

Profile Model — the candidate's full professional identity:

```
userId (reference to User), personal info, educations[], experiences[],
projects[], skills{languages, frameworks, tools, libraries},
certs[], awards[], leaders[], volunteers[], pubs[],
extras, consent, githubCache{repos, fetchedAt}, isComplete
```

One Profile per candidate. The `githubCache` is particularly smart — it stores fetched GitHub repos so every dashboard visit doesn't spend a GitHub API call.

Roadmap Model — the AI-generated career plan:

```
userId (reference to User), current (full roadmap object), previous (full roadmap
object)
```

Only two slots ever exist per user (current and previous). Generating a new roadmap rotates them. The previous slot acts as an "undo." There's no history beyond two. This is a deliberate simplicity choice — no need for version history, just the latest and the one before.

Job Model — the job posting with embedded applications:

```
companyId (reference to User), title, department, employmentType,
workplaceType, location, jobSummary, responsibilities, requirements,
skills[], salary, perks[], deadlines, attachments (PDF URLs),
status ('draft'/'active'/'closed'), applications[]
```

The `applications[]` array is embedded inside each Job document. Each entry contains:

```
candidateId, appliedAt, status
('new'/'reviewed'/'shortlisted'/'interview'/'hired'/'rejected'),
phone, relocate, noticePeriod, pitch, topChoice, followCompany, resumeSource,
resumeUrl
```

Why embed applications inside jobs? The most common query is "get this job's applicants" — with embedding, that's a single document read. If applications were in a separate collection, you'd need a join/lookup. The trade-off: job documents can get large if a job has thousands of applications, but for this scale that's acceptable.

OtpRecord Model (in `otp.js`):

```
email, purpose ('signup'/'forgot-password'), codeHash (bcrypt), expiresAt
```

Has a MongoDB **TTL index** on `expiresAt` — MongoDB automatically deletes expired OTP documents. No cron job needed; the database itself manages cleanup.

---

8. The OTP System — How Email Verification Works

The OTP (One-Time Password) system is entirely custom-built, not relying on any third-party OTP service:

1. **Generate** a cryptographically random 6-digit number using `crypto.randomInt()`
2. **Hash it** with bcrypt (10 rounds) and store the hash in MongoDB with an expiry timestamp
3. **Email the plain code** to the user via Zoho SMTP using Nodemailer
4. **On verification**: find the record in MongoDB → check not expired → `bcrypt.compare(submittedCode, storedHash)` → if match, delete the record and proceed
5. **MongoDB TTL index** auto-deletes expired records

The OTP email itself is a **beautiful branded HTML email** — not a plain text message. It has a gradient header, individually colored digit cells, and an expiry notice. This is built entirely as an HTML string in `otp.js`.

Why hash OTPs? If the database were compromised, an attacker would see hashed codes, not the 6-digit numbers. Combined with the 10-minute expiry, this makes OTP database leaks essentially useless.

---

9. The AI Systems — Theory Behind Each Integration

9.1 Career Roadmap via Groq (Llama 3.3 70B)

What is Groq? Groq is an AI inference provider — it runs large language models (LLMs) at extremely high speed using custom hardware. The model used is Meta's **Llama 3.3 70B** (70 billion parameters), an open-source LLM.

How does the roadmap generation work?

The backend sends Groq a **system prompt** — a long, carefully engineered instruction set that tells the model exactly what to produce. The system prompt specifies:

- Generate exactly 4 phases (Foundational, Intermediate, Advanced, Mastery)
- Each phase must have exactly these fields
- Topics: 6-10 items, Projects: 2-3, Skills: 6-10, Resources: exactly 3
- Return ONLY valid JSON, no explanation, no markdown code blocks
- Allowed color values: 'blue', 'purple', 'green', 'orange'
- Allowed level values: 'Foundational', 'Intermediate', 'Advanced', 'Mastery'

The user's target role (e.g., "Machine Learning Engineer") is sent as the user message.

Why JSON mode? The response is set with `response_format: { type: 'json_object' }` — this tells Groq to force the model to output valid JSON. Without this, the model might wrap its answer in markdown code blocks or add explanation text.

Validation and retry logic: Even with JSON mode, the model occasionally produces JSON that doesn't match the expected schema (wrong phase count, wrong topic count, invalid level strings). The backend **validates the output** against strict rules. If validation fails, it sends the error back to the model with a correction request and tries again — up to 3 attempts. This "self-correction" pattern is common in production LLM systems.

Rotation strategy: Two slots (`current` and `previous`) are maintained. When you generate a new roadmap, the current one slides into `previous` and the new one becomes `current`. Calling "load previous" swaps them. This means "load previous" twice in a row is a no-op — it's a toggle, not a history stack.

9.2 Job Recommendations via Gemini 2.0 Flash

What is Gemini? Google's family of LLMs. The `gemini-2.0-flash` model is a fast, cost-effective variant suited for structured tasks.

What does it do here? When a candidate views the Jobs page, the backend can send the candidate's skills (from their profile) along with available job listings to Gemini and ask it to **rank the jobs by relevance**. The model returns a JSON array of job IDs ordered by how well they match the candidate's skill set.

Same retry pattern: one automatic retry if the JSON is invalid, feeding the parse error back to the model.

---

10. The GitHub Integration — Caching Theory

When a candidate adds their GitHub username to their profile, the backend can fetch their repositories from the GitHub API. The challenge: the GitHub API is rate-limited (60 requests/hour without authentication, 5000/hour with a token). If every user loading the dashboard triggered a GitHub API call, the rate limit would be exhausted quickly.

The solution: Cache the repos inside the profile document (`githubCache.repos` and `githubCache.fetchedAt`).

When the dashboard loads and requests repos:

1. Backend checks if `githubCache.repos` is populated and `fetchedAt` exists
2. If yes → return the cached repos instantly (no GitHub API call)
3. If no → call GitHub API, store in cache, return

The only way to refresh the cache is the explicit "Refresh GitHub" button on the dashboard, which calls a different endpoint (`POST /api/github/repos/refresh`) that always hits the live GitHub API.

This **cache-aside pattern** keeps dashboard loads fast while ensuring the data can be refreshed on demand.

---

11. File Uploads — How Multer Works

Files (profile photos, certificate PDFs, resume uploads, job description PDFs) are stored locally on the server using **Multer**, an Express middleware for handling `multipart/form-data` requests.

Multer sits between the route and the controller. When a request with a file arrives:

1. Multer reads the file bytes from the request
2. Based on the **storage configuration** (destination folder, filename template), it writes the file to disk
3. It populates `req.file` or `req.files` with metadata (filename, path, size, mimetype)
4. The controller then saves the file path to the database

Different upload configurations for different purposes:

- Profile photos → `uploads/photos/` (images only, 5MB max)
- Certificate PDFs → `uploads/certs/` (PDFs only)
- Job PDFs → `uploads/jobs/` (PDFs only, 5MB max)
- Resume uploads → `uploads/resumes/` (PDF/DOC/DOCX, 2MB max)

Files are served back to the browser via the `/uploads` static middleware in `app.js`.

The reason for separate folders per purpose is **organization and access control** — in a future enhancement, access to `uploads/resumes/` could be restricted to companies who have a specific application from that candidate.

---

12. The Hiring Pipeline — State Machine Theory

The hiring pipeline is conceptually a **state machine** — each application exists in exactly one state, and transitions between states follow defined rules:

```
new → reviewed → shortlisted → interview → hired
                                         ↘ rejected (from any active state)
```

The `status` field in each application entry in the Job document holds the current state. The company can move a candidate forward or mark them as rejected from any stage.

Why embed the pipeline in the job document?

- "All applicants for this job" = read one document
- "Move this candidate to the next stage" = update one field in one document
- No complex joins between a jobs collection and a separate applications collection

Privacy protection: The `toCandidateView()` method on the Job model strips the entire `applications[]` array before sending job data to candidates. A candidate can only see their own application status — never other candidates' data.

---

13. The "Profile-Completed Gate" — Why Candidates Are Forced Through the Builder

When a candidate signs up (via email or OAuth), their `profileCompleted` flag in the User document is `false`.

Every protected route for candidates (Dashboard, Jobs, Roadmap, Profile) is wrapped in `ProtectedRoute`. This guard checks:

```
if (user.role === 'candidate' && !user.profileCompleted && location.pathname !==
'/profile-builder')
  → redirect to /profile-builder
```

This means **new candidates literally cannot access any other page** until they complete the profile builder. The profile builder saves `isComplete: true` when submitted, which triggers the backend to also set `user.profileCompleted = true`.

Why this design?

- Ensures companies always see fully filled-out profiles (no empty applications)
- Forces candidates to think about their qualifications before applying
- Guarantees the AI roadmap has enough data to work with (skills are required in the builder)

---

14. Layouts and the Persistent Sidebar Pattern

The `CandidateLayout` and `CompanyLayout` components are **shell components** — they render the Sidebar + Topbar, and then an `<Outlet/>` where child pages render.

In React Router v6+, when a parent route's element doesn't change (the layout stays the same) and only the child route changes (Dashboard → Jobs), the layout component is **not re-mounted**. The Sidebar and Topbar components stay alive and in the same DOM position.

Without this pattern, every page navigation would unmount and remount the sidebar — causing visual flickering, re-running sidebar setup effects, and losing any sidebar-local state (like which section is expanded).

This is the "persistent layout" pattern — widely used in dashboard applications for a smooth navigation experience.

---

15. The Application Form — Two Resume Sources

When a candidate applies to a job, the Application Modal (`ApplicationModal.jsx`) lets them choose how they submit their resume:

Option 1: Share SkillSphere Profile The candidate's already-filled profile (education, experience, projects, skills) is shared with the company. The application stores `resumeSource: 'profile'` and the company can click to view the candidate's full SkillSphere profile.

Option 2: Upload Resume File The candidate uploads a PDF/DOC/DOCX file. It's stored in `uploads/resumes/`. The application stores `resumeSource: 'upload'` and `resumeUrl` pointing to the file.

This dual approach makes sense because:

- New users or those on different platforms may have a resume already prepared
- SkillSphere profile sharing is the "native" path that gives companies the richest data

The company can view either: clicking a profile-sourced application shows the SkillSphere profile viewer; clicking an upload-sourced application provides a link to download the file.

16. Security Summary

Threat	Mitigation
Password theft	Firebase handles credentials; SkillSphere never stores passwords
Token theft	Access tokens are short-lived (7 days); refresh tokens are hashed in DB
Brute-force login	Auth rate limiter (30 req/15min per IP)
DDoS	Global rate limiter (600 req/15min)
Fake email signup	OTP email verification before account creation
OTP database leak	OTPs stored bcrypt-hashed, expire in 10 minutes
Candidate privacy	Job `toCandidateView()` strips all other applicants' data
Role confusion	`authorize('candidate')` / `authorize('company')` middleware enforces role on every route
XSS via file uploads	Multer validates MIME types; only allowed file types stored
Clickjacking / header attacks	Helmet sets all security headers automatically

17. How Everything Connects — The Complete User Journey

Candidate Journey (First Time)

```
1. Visits skillsphere.com
   → Sees HomePage (marketing)
   → Clicks "Get Started"

2. GetStartedPage
   → Types email, selects "Candidate"
   → Clicks "Send OTP"
   → [Backend: checks email → generates OTP → hashes it → saves to DB → emails via
Zoho]
   → Gets 6-digit code in email
   → Types OTP
   → [Backend: finds OTP record → bcrypt.compare → deletes record]
   → Types name + password
   → Clicks "Create Account"
   → [Backend: creates Firebase user → creates MongoDB User
(profileCompleted=false) → issues JWT tokens]
   → Frontend stores tokens, sets user in AuthContext

3. ProtectedRoute redirects to /profile-builder
   (profileCompleted is false)

4. ProfileBuilderPage
```

```
   → Fills out all sections over multiple minutes (auto-saves every 30s)
   → Clicks "Complete Profile"
   → [Backend: saves Profile doc (isComplete=true) → sets
user.profileCompleted=true]
   → Frontend navigates to /dashboard/candidate

5. StudentDashboardPage loads
   → Fetches GitHub repos (live first time, cached after)
   → Shows profile completion progress
   → Shows roadmap stub ("Generate your first roadmap")
   → Shows recent job listings

6. Goes to /roadmap
   → Types "Full Stack Developer"
   → [Backend: calls Groq → Llama 3.3 70B generates 4-phase roadmap → validates →
saves to DB]
   → Roadmap renders with 4 phase cards

7. Goes to /jobs
   → Browses job listings
   → Clicks a job, reads details
   → Clicks "Apply"
   → Application Modal opens
   → Chooses "Share SkillSphere Profile"
   → Answers pitch questions
   → Submits
   → [Backend: adds application entry to Job.applications[], applicantsCount++]
   → Modal closes, job card shows "Applied"
```

Company Journey (First Time)

```
1. Signs up as Company on GetStartedPage
   → Same OTP flow, chooses "Company" role, types company name

2. No profile-builder for companies
   → profileCompleted defaults to true for companies (or the gate is candidate-
only)
   → Redirected to /dashboard/company

3. CompanyDashboardPage
   → Stats all zero (new account)
   → Navigates to /postings

4. PostJobPage
   → Fills out job form (title, description, skills, salary, etc.)
   → Clicks "Save as Draft" first (status='draft', not visible to candidates)
   → Reviews, clicks "Publish" (status='active', visible to candidates)
   → [Backend: validates completeness → sets publishedAt → job is now live]

5. Candidates start applying
```

```
6. /applications or /candidates
   → Sees applicant list
   → Clicks a candidate → views their SkillSphere profile or downloaded resume
   → Changes status from 'new' → 'shortlisted' → 'interview'
   → Eventually marks as 'hired' or 'rejected'
```

---

18. Summary of Key Theoretical Concepts Used

Concept	Where Applied
SPA (Single Page Application)	The entire React frontend
Component-based UI	Every page/component is a reusable React function
Context API (Global State)	AuthContext, JobsContext, RoadmapContext
Persistent Layout Pattern	CandidateLayout / CompanyLayout with `<Outlet/>`
Route Guards	ProtectedRoute, role-based access control
REST API	The entire Express backend
Middleware Pipeline	Helmet, CORS, auth, rate limiting, validation
MVC-style layering	Routes → Controllers → Services
Dual Authentication	Firebase (credentials) + JWT (API sessions)
Token Refresh Flow	Silent re-authentication on 401
Document Database	MongoDB with embedded sub-documents
TTL Index	Auto-expiring OTP records in MongoDB
Cache-aside Pattern	GitHub repos cached in Profile document
State Machine	Hiring pipeline stages (new→reviewed→...→hired)
Progressive Disclosure	Profile builder wizard with sections
Prompt Engineering	Detailed system prompt for Groq/Gemini
LLM Self-Correction	Retry loop with error feedback to the model
Multipart File Upload	Multer for photos, PDFs, resumes
Rate Limiting	Global + auth-specific request throttling
bcrypt Hashing	OTPs, refresh tokens never stored in plaintext
Security Headers	Helmet middleware
CORS	Controlled cross-origin access